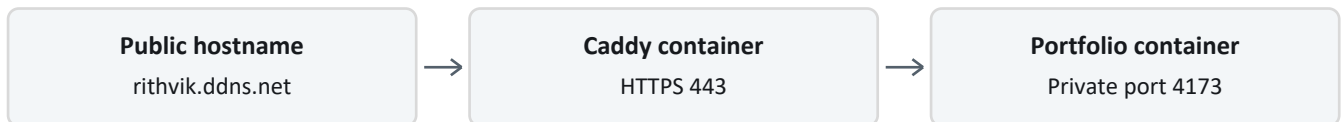


Portfolio Docker and Caddy Guide

Local development and Oracle deployment

Configuration recorded on 7 October 2026

Use Docker Compose on your computer to preview the portfolio. On Oracle, use the redeployment script after your changes reach **main**. This guide explains the files, commands, saved settings, and steps for routing additional sites through Caddy.



DNS points the hostname to **144.24.135.201**. Caddy accepts public HTTP and HTTPS traffic and forwards website requests to the Node application. The application uses your SMTP provider for contact emails.

Where	Configuration
Your computer	Docker Desktop; local URL http://localhost:8080
Oracle server	SSH user opc ; checkout /opt/portfolio ; deployment branch main
Live website	https://rithvik.ddns.net/ ; Caddy certificates persist in Docker volumes

Run locally

From the repository folder in PowerShell, with Docker Desktop running:

```
docker compose up --build -d --wait
```

Open **http://localhost:8080**. Your computer's port 8080 forwards to port 4173 inside the container. Repeat the command after changing content or code; this setup does not rebuild automatically while you edit.

Deploy to Oracle

Commit your development changes, merge them into **main**, and push. Then run this in the Oracle SSH session:

```
bash /opt/portfolio_redeploy.sh
```

Before the first normal run, commit and merge the deployment files prepared during setup. The script stops if server edits differ from the pushed versions. Pushing a development branch by itself does not update the live site.

Understand the three Docker terms

The **Dockerfile** is the build recipe. An **image** is the packaged application produced by that recipe. A **container** is a running instance of an image. Compose describes how images and containers are built, connected, and started.

Files and environment settings

Choose one Compose file for the environment you are running. The local, IP preview, and production configurations are separate entry points.

File	Purpose and use
Dockerfile	Builds the portfolio image, installs dependencies, generates pages, checks links, and runs tests. The final image contains the runtime application.
compose.yaml	Local preview on localhost:8080. Loads the optional local .env and runs one application container.
compose.ip.yaml	Earlier HTTP preview on the public IP. Publishes port 80 directly to the app and does not load SMTP credentials.
compose.production.yaml	Current Oracle setup. Runs the app and Caddy, supplies production settings, keeps the app port private, and preserves certificates.
.dockerignore	Allowlist of build inputs. Keeps credentials and unnecessary files out of the image. Add required new assets to this list.
deploy/Caddyfile.docker	Active proxy configuration. Mounted inside Caddy as /etc/caddy/Caddyfile. Maps hostnames to applications.
deploy/Caddyfile deploy/portfolio.service	Templates for the alternative setup that runs Caddy and Node directly on Linux. They are not used by the Docker deployment.
deploy/DOCKER.md	Repository guide for Docker setup, updates, environment settings, and operation.

Keep settings separate from the image

Settings file	Where it applies
.env	On your computer. Supplies local settings and any local SMTP credentials.
.env.production	On Oracle, at /opt/portfolio/.env.production. Holds DOMAIN, SMTP settings, and Docker network values.
.env.ip	Used only with the earlier public-IP preview.
*.env.example	Templates to copy and fill in. They contain placeholders, not your working credentials.

The server reads its own saved **.env.production** at runtime. Keep it private and untracked by Git. Your local .env is not uploaded by a Git push. If credentials change, update the server file separately and recreate the app through Compose.

Scripts and everyday commands

Script or command	What it does
<code>/opt/portfolio_redeploy.sh</code>	Oracle shortcut that invokes the versioned script in <code>/opt/portfolio/portfolio_redeploy.sh</code> .
<code>portfolio_redeploy.sh</code>	Fetches main, checks server edits, validates configuration, builds app, updates containers, reloads Caddy, and checks HTTPS and the contact API.
<code>scripts/build.mjs</code>	Generates the HTML pages and sitemap from content and templates. Production <code>SITE_URL</code> comes from <code>DOMAIN</code> during the image build.
<code>scripts/check.mjs</code>	Checks pages, local links, anchors, and the CV file.
<code>pnpm test</code>	Runs the application tests. The Docker build also runs these tests.
<code>pnpm dev</code>	Runs Node directly on your computer and loads <code>.env</code> . This is an alternative to the Docker preview.
<code>scripts/serve.mjs</code>	Older preview entry point. It does not automatically load <code>.env</code> .
<code>oracle-micro-amd-login.bat</code>	Opens SSH using the key beside the batch file. Login alone does not deploy the application.

Local status and logs

```
docker compose ps
docker compose logs --tail=100 app
docker compose down
```

The last command stops and removes the local container. Run the local up command to start it again. A build packages the app; it does not by itself start a container.

Oracle status and logs

The following helper shortens production commands in the current SSH shell:

```
cd /opt/portfolio
dc() {
  sudo docker compose --env-file .env.production \
    -f compose.production.yaml "$@"
}
dc ps
dc logs --tail=100 app caddy
```

To apply server environment changes without fetching Git changes:

```
bash /opt/portfolio_redeploy.sh --no-pull
```

Compose **up** recreates containers when their configuration changes. A plain **restart** does not load new environment values or rebuild code. Docker can reuse unchanged build steps from its cache.

Add another site through Caddy

Use one public Caddy instance for the Oracle IP. Give each additional application a container and a hostname. The example below adds a service named **tools** on internal port **3000**; substitute the real name, port, source folder, and domain.

Create the DNS record

Create an A record for **tools.yourdomain.com** pointing to **144.24.135.201**. This must be a hostname you control. An explicit record is enough; wildcard DNS is optional. The existing rithvik.ddns.net record alone does not create tools.rithvik.ddns.net.

Connect the new application to Caddy

This example adds the app to the existing Compose project. Merge these entries into compose.production.yaml; retain the current app, Caddy image, ports, volumes, environment, and backend network definition. The new app must listen on 0.0.0.0:3000 inside its container.

```
services:
  tools:
    build: ../my-tools
    restart: unless-stopped
    networks: [tools_net]
  caddy:
    networks:
      backend:
        ipv4_address: ${CADDY_IP:-172.30.0.3}
        tools_net: {}

networks:
  tools_net: {}
```

The ../my-tools folder must exist on Oracle and contain that app's Dockerfile. The portfolio script does not fetch another repository. If the tool has its own repository or Compose project, give it a separate update process.

Add the hostname to the Caddyfile

Add this block alongside the existing blocks in deploy/Caddyfile.docker:

```
tools.yourdomain.com {
  encode zstd gzip
  reverse_proxy tools:3000
}
```

Caddy uses the Docker service name and the app's internal port. Inside the Caddy container, localhost means Caddy itself. A shared Docker network lets Caddy reach the app without publishing port 3000 on Oracle. [1, 2]

Once DNS works and Caddy is reachable on ports 80 and 443, it obtains and renews a certificate for the new hostname automatically. The existing certificate volumes remain in use. [3]

Apply proxy changes and change domains

Start the additional site

After the new code and configuration are on Oracle, use the **dc** helper from page 3 in that SSH shell:

```
dc config --quiet
dc build tools
dc up -d --no-deps tools
dc up -d --no-deps --force-recreate caddy
dc logs --tail=100 tools caddy
```

Check the new site over HTTPS after both services start. Recreating Caddy briefly restarts the proxy while keeping certificate storage. This also refreshes the single-file bind mount when Git or an editor has replaced the Caddyfile.

The current redeployment script explicitly builds **app**. To make it build every service with a build definition in this Compose project, change its build command from **build app** to **build**. Add checks for each new site; the existing public checks cover the portfolio only.

Validate and reload an existing Caddy configuration

For a file already visible inside the running container, use:

```
dc exec -T caddy caddy validate \
--config /etc/caddy/Caddyfile --adapter caddyfile
dc exec -T caddy caddy reload \
--config /etc/caddy/Caddyfile --adapter caddyfile
```

Reload applies proxy configuration without rebuilding the Node app. If the container still sees an older copy after a Git update, recreate Caddy as shown above. A network or container environment change also requires Compose to recreate the affected container.

Replace the No IP hostname with a purchased domain

Point the purchased domain's A record at 144.24.135.201. Then edit the private production settings on Oracle and set DOMAIN to the bare hostname, without https:// or a trailing slash:

```
cd /opt/portfolio
nano .env.production
# Example setting inside that file:
# DOMAIN=yourdomain.com
bash /opt/portfolio_redeploy.sh --no-pull
```

The rebuild updates canonical URLs and the sitemap. Compose updates the app's allowed origin and Caddy's DOMAIN; Caddy requests the new certificate. Add an explicit DNS record and Caddy block for www if you want that address too.

Put a tool under the existing hostname

A URL such as rithvik.ddns.net/tools/ uses the existing DNS record and certificate. Caddy can route that path to another app, but the app must support the /tools/ base path, including asset URLs and redirects. Separate hostnames are generally easier for independent apps. [4]

Checks and troubleshooting

Symptom or task	What to check
Local port 4173 does not open	Use <code>http://localhost:8080</code> for Docker. Port 4173 is inside the container; the direct Node preview uses a different setup.
Code changed but the page did not	Rebuild and run locally with <code>up --build</code> . On Oracle, push main and run the redeployment script. A Git pull alone does not change a running image.
The script stops on server edits	Compare the reported file with <code>origin/main</code> . Commit and merge the intended local changes or reconcile the server file. The script backs up matching copied files before updating.
The contact API says available false	Check the required SMTP and contact settings in the server's <code>.env.production</code> , then recreate the app. Do not paste credentials into shared logs or Git.
The API says available true but mail fails	Availability means settings are present. Check SMTP authentication, connectivity, and provider responses. Check the receiving inbox and spam folder after submitting a message yourself.
A new site returns 502	Check that its container is running, listens on the expected internal port, and shares a network with Caddy. Check the service name in <code>reverse_proxy</code> .
HTTPS does not become available	Check DNS, Oracle and VM firewall access on TCP 80/443, and Caddy logs. UDP 443 is optional for HTTP/3.
Stop production temporarily	Use <code>dc down</code> . Keep the named volumes; <code>down --volumes</code> deletes stored certificate state.

Public checks

```
curl -I https://rithvik.ddns.net/  
curl https://rithvik.ddns.net/api/contact
```

At setup verification, the site returned HTTP 200, the contact API returned available true, and SMTP authentication succeeded. No test email was sent by the deployment process. These are recorded checks, not a continuing availability guarantee.

Reference documentation

1. Caddy with Docker Compose
<https://caddyserver.com/docs/running#docker-compose>
2. Docker Compose networking
<https://docs.docker.com/compose/how-tos/networking/>
3. Caddy automatic HTTPS
<https://caddyserver.com/docs/automatic-https>
4. Caddy path routing
https://caddyserver.com/docs/caddyfile/directives/handle_path

Repository reference: `deploy/DOCKER.md`. Recheck configuration after changing domains, service ports, Docker networks, or deployment scripts.